

Lab #03

Declaration of classes with methods and parameters, loops and Value Types vs. Reference Types.

Overview of Classes

A Class Is an Active Data Structure

A class is a data structure that can store data and execute code. It contains the following:

- **Data members**, which store data associated with the class or an instance of the class. Data Members generally model the attributes of the real-world object the class represents.
- **Function members**, which execute code. Function members generally model the functions and actions of the real-world object the class represents.

A C# class can have any number of data and function members. The members can be any Combination of nine possible member types. These member types are shown in Table 4-1.

Table 4-1. *Types of Class Members*

Data Members–Store Data	Function Members–Execute Code	
✓ Fields	✓ Methods	☐ Operators
☐ Constants	☐ Properties	☐ Indexers
	☐ Constructors	☐ Events
	☐ Destructors	

Declaring a Class

The class declaration provides the following:

- The class name
- The members of the class
- The characteristics of the class

```

Keyword  Class name
  ↓       ↓
class MyExcellentClass
{
    MemberDeclarations
}
```

```

class Dealer // Class declaration
{
    ...
}

class Player // Class declaration
{
    ...
}
```

Note: Class declaration referred to as a “class definition”

Class Members

Fields and methods are the most important of the class member types. Fields are data members and methods are function members.

Fields

A field is a variable that belongs to a class.

- It can be of any type, either predefined or user-defined.
- Like all variables, fields store data, and have the following characteristics:

```
Type  
↓  
Type Identifier;  
↑  
Field name
```

```
class MyClass  
{ Type  
  ↓  
  int MyField;  
  ↑  
  Field name  
}
```

Methods

A method is a named block of executable code that can be executed from many different parts of the program, and even from other programs. When a method is called, or invoked, it executes its code, and then returns to the code that called it.

Return type: This states the type of value the method returns. If a method does not return a value, the return type is specified as void.

Name: This is the name of the method.

Parameter list: This consists of at least an empty set of matching parentheses. If there are parameters, they are listed between the parentheses.

Method body: This consists of a matching set of curly braces, containing the executable code.

```
class SimpleClass  
{  
  Return type    Parameter list  
  ↓             ↓  
  void PrintNums ( )  
  {  
    Console.WriteLine("1");  
    Console.WriteLine("2");  
  }  
}
```

Class

A class is simply a representation of a type of object. It is the blueprint/ plan/ template that describe the details of an object. A class is the blueprint from which the individual objects are created. Class is composed of three things: a name, attributes, and operations.

Declaring Classes

```
Public class Employee
{
    //Fields, properties, methods and events go here...
}
```

Object Creation

```
Employee secretary;           // declares secretary
secretary = new Employee ();  // allocates space
```

```
Employee secretary = new Employee(); // does both
```

```
But the secretary is still "blank"
secretary.name = "Adele";        // dot notation
secretary.birthday ();           // sends a message
```

Object-creation expression

The keyword new

```
Dealer theDealer;           // Declare variable for the reference.
theDealer = new Dealer();   // Allocate memory for the class object.
               ↑
            Object-creation expression
```

Example # 01

```
using System;
namespace example1
{
    class example1
    {
        public int v = 1;
    }

    class Program
    {
```

```
static void Main(string[] args)
```

```
{
    example1 e;
    e = new example1();
    Console.WriteLine("Value of integer is "+e.v);
    Console.ReadKey();
}
}
```

Example # 02

```
using System
namespace example2
{
    class temp
    {
        public float high;
        public float low;

        public float average()
        {
            return ((high + low) / 2);
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            temp tobj = new temp();
            tobj.high = 40;
            tobj.low = 10;

            Console.WriteLine("Low Temp is " + tobj.low);
            Console.WriteLine("High Temp is " + tobj.high);
            Console.WriteLine("Average Temp is " + tobj.average());
            Console.ReadKey();
        }
    }
}
```

C# Conditions and If Statements

C# supports the usual logical conditions from mathematics:

- Less than: $a < b$
- Less than or equal to: $a \leq b$
- Greater than: $a > b$
- Greater than or equal to: $a \geq b$
- Equal to: $a == b$
- Not Equal to: $a != b$

You can use these conditions to perform different actions for different decisions.

C# has the following conditional statements:

- Use if to specify a block of code to be executed, if a specified condition is true
- Use else to specify a block of code to be executed, if the same condition is false
- Use else if to specify a new condition to test, if the first condition is false
- Use switch to specify many alternative blocks of code to be executed

Example # 03

using System;

```
namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            if (20 > 18)
            {
                Console.WriteLine("20 is greater than 18");
            }
        }
    }
}
```

Example # 04

```
using System;

namespace MyApplication
{
    class Program
    {

        static void Main(string[] args)
        {
            int x = 20;
            int y = 18;
            if (x > y)
            {
                Console.WriteLine("x is greater than y");
            }
        }
    }
}
```

C# Switch Statements

This is how it works:

- The switch expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.
- The break and default keywords will be described later in this chapter

The example below uses the weekday number to calculate the weekday name:

Example # 05

```
using System;

namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            int day = 4;

            switch (day)
            {
                case 1:
                    Console.WriteLine("Monday");
                    break;
                case 2:
                    Console.WriteLine("Tuesday");
                    break;
                case 3:
                    Console.WriteLine("Wednesday");
                    break;
                case 4:
                    Console.WriteLine("Thursday");
                    break;
                case 5:
                    Console.WriteLine("Friday");
                    break;
                case 6:
                    Console.WriteLine("Saturday");
                    break;
                case 7:
                    Console.WriteLine("Sunday");
                    break;
            }
        }
    }
}
```

C# While Loop

Loops can execute a block of code as long as a specified condition is reached.

Loops are handy because they save time, reduce errors, and they make code more readable.

The while loop loops through a block of code as long as a specified condition is True:

Example # 06

```
using System;

namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {

            int i = 0;
            while (i < 5)
            {
                Console.WriteLine(i);
                i++;
            }
        }
    }
}
```

C# for Loop

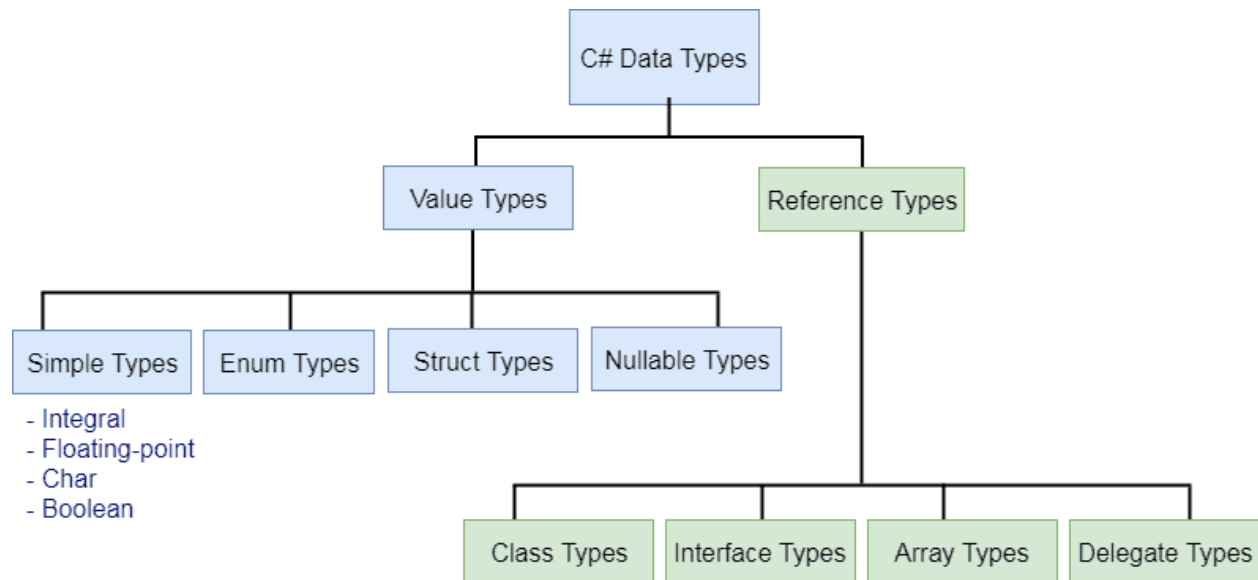
Example # 07

```
using System;

namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 0; i < 5; i++)
            {
                Console.WriteLine(i);
            }
        }
    }
}
```

C# VALUE TYPE VS REFERENCE TYPE

C# mainly categorized data types in two types: Value types and Reference types. Value types include simple types (such as int, float, bool, and char), enum types, struct types, and Nullable value types. Reference types include class types, interface types, delegate types, and array types.



IMPLICIT

```
int myInt = 9;

double myDouble = myInt;    // Automatic casting: int to double

Console.WriteLine(myInt);   // Outputs 9
Console.WriteLine(myDouble); // Outputs 9
```

EXPLICIT

```
double myDouble = 9.78;

int myInt = (int) myDouble; // Manual casting: double to int

Console.WriteLine(myDouble); // Outputs 9.78
Console.WriteLine(myInt);    // Outputs 9
```

Task:

EVERY CODE SHOULD HAVE YOUR ROLL NUMBER AND NAME.

1. Write the output of the following program.
using System;

```
public class Book
{
    public string Title;
    public string Author;
    public short YearPublished;
    public int NumberOfPages;
    public char CoverType;
}

public class Exercise
{
    static void Main()
    {
        var First = new Book();

        First.Title = " Turbo C ";
        First.Author = "Robert Lafore";
        First.YearPublished = 1996;
        First.NumberOfPages = 872;
        First.CoverType = 'H';
        Console.WriteLine("Book Characteristics");

        Console.Write("Title: ");
        Console.WriteLine(First.Title);
        Console.Write("Author: ");
        Console.WriteLine(First.Author);
        Console.Write("Year: ");
        Console.WriteLine(First.YearPublished);
        Console.Write("Pages: ");
        Console.WriteLine(First.NumberOfPages);
        Console.Write("Cover: ");
        Console.WriteLine(First.CoverType);
        Console.ReadKey();
    }
}
```

2. Create a class student with a data members name, age, marks of English , marks of math, marks of science, total marks, obtained marks and percentage provide member functions CalculateTotalMarks and CalculatePercentage to calculate marks and percentage in main.
3. Write a program that contains a class which has a method that takes user name as input and second functions which returns number of vowels present in it and Main program prints the number of vowels
4. Write a program that performs typecasting of atleast 3 predefined datatypes in C#.
5. Write a C# program that takes a number as input and displays it four times in a row (separated by blank spaces), and then four times in the next row, with no separation. You should do it twice: Use the console. Write and use {0}.

Test Data:

Enter a digit: 25

Expected Output:

25 25 25 25

25252525

25 25 25 25

25252525